

SkillTwin

Master Implementation & Product Plan

Evidence-Based Skill Development Operating System

A single working blueprint for the complete product, architecture, MVP and phased implementation.

1. Product Definition

SkillTwin is an evidence-based career-readiness system that analyzes what a student has actually demonstrated, compares it with the skills required for a target role, identifies and prioritizes gaps, creates a personalized path to close those gaps, verifies newly demonstrated skills, and continuously updates the student's SkillTwin.

Core philosophy: “Don’t just tell us what skills you have. Show us the evidence.”

2. Complete Product Loop

STUDENT → ONBOARDING → COLLECT EVIDENCE → EVIDENCE ENGINE → SKILL EXTRACTION → SKILL NORMALIZATION → SKILL TWIN → TARGET ROLE → INDUSTRY REQUIREMENTS → SKILL GAP ANALYSIS → PRIORITY ENGINE → PERSONALIZED ROADMAP → LEARN + PRACTICE → BUILD → VERIFY → UPDATE SKILL TWIN → CAREER READINESS → REPEAT

Operational loop: **Discover → Measure → Analyze → Prioritize → Learn → Practice → Build → Validate → Verify → Update → Repeat.**

3. Problem We Are Solving

Students may have skills on resumes, projects on GitHub, certificates, self-declared knowledge and academic experience, but often lack an evidence-based system that answers: **What can this student actually demonstrate, how strong is the evidence, and how does it compare with what their target job requires?**

SkillTwin goes beyond listing missing skills. It explains the evidence, identifies the gap, prioritizes it, gives a path to close it, verifies the result, and updates the profile.

4. Product Architecture

- Student → Onboarding → Evidence Collection
- Resume + GitHub → Evidence Engine
- Evidence → Skill Extraction → Skill Normalization
- Normalized Evidence → Living SkillTwin
- SkillTwin + Target Role → Industry Requirements
- Industry Requirements + SkillTwin → Gap Analysis
- Gap Analysis → Priority Engine → Personalized Roadmap
- Roadmap → Learn → Practice → Build
- Built Project → GitHub Verification
- Verification → Updated SkillTwin → Recalculated Career Readiness

5. Onboarding

Collect the information needed to personalize the rest of the system.

- Name

- Email
- Education level
- Degree
- Branch
- Semester/year
- Target role
- Study time per day
- Preferred learning style
- Preferred language

6. Evidence Collection

Resume

Upload PDF or DOCX. Extract skills, technologies, education, experience, projects, certifications, roles and relevant descriptions.

GitHub

Connect a GitHub profile and analyze repositories. The MVP should use repository evidence rather than trusting self-declared skills.

Future Evidence Sources

The architecture can later support assessments, certifications, practical projects and other evidence, but the MVP remains focused.

7. Evidence Engine

Convert raw artifacts into structured, traceable evidence. Distinguish a simple skill mention from evidence of actual implementation.

Example: “Built a REST API using FastAPI and PostgreSQL” can support evidence for Python, FastAPI, REST APIs, PostgreSQL and backend development, but the system must not automatically assume advanced expertise.

Evidence should retain its source and reasoning so SkillTwin results can explain why a skill was assigned.

8. Skill Normalization

Map different names for the same technology to a canonical skill.

- JS / JavaScript / ECMAScript → JavaScript
- React.js / React JS / React → React

Normalization creates a consistent internal skill vocabulary.

9. Living SkillTwin

Every skill is represented through proficiency, confidence, evidence and reasoning rather than one unexplained score.

Dimension	Meaning
Proficiency	Beginner / Intermediate / Advanced
Confidence	How strongly the evidence supports the proficiency estimate

Evidence	Resume, GitHub, Project, Assessment, etc.
Reasoning	Human-readable explanation connecting evidence to the assigned level

React Proficiency: Intermediate Confidence: 82% Evidence: Resume ✓, GitHub ✓, Project ✗, Assessment ✗
Reason: GitHub evidence shows React repositories with component-based UI and API integration, but insufficient evidence of advanced state management or testing.

10. Target Role

The student chooses the career direction against which the SkillTwin will be evaluated.

- Full-Stack Developer
- Frontend Developer
- Backend Developer
- Software Engineer
- Data Analyst
- ML Engineer

For the MVP, use a small curated set of roles.

11. Industry Requirement Engine

Each target role has curated required skills, expected levels and importance. The MVP should not pretend to represent every company or scrape the entire live job market.

Example: Full-Stack Developer → JavaScript (Advanced/Critical), React (Intermediate/Critical), Node.js (Intermediate/High), SQL (Intermediate/High), Docker (Basic/Medium).

12. Skill Gap Analysis

- React: Beginner → Required Intermediate → Moderate gap
- Node.js: Beginner → Required Intermediate → High gap
- Docker: Missing → Required Basic → Critical gap

Every gap includes an explanation of why it exists and why it matters for the selected role.

13. Career Readiness / Industry Alignment

Provide an overall, explainable readiness/alignment view. Ground it in current proficiency, required proficiency, skill importance and evidence confidence so the result is not a magic number.

14. Priority Engine

Prioritize gaps instead of presenting an unstructured list. Consider role importance, gap size, dependencies and career impact.

Example: 1) Node.js – Critical, 2) React – High, 3) Testing – High, 4) Docker – Medium, 5) Kubernetes – Later.

15. Personalized Roadmap

- Why the skill matters for the target role
- Learning topics
- Curated learning resources
- Practice exercises

- Recommended project
- Proof / verification step

The roadmap should answer: **What exactly should I do next?**

16. Learning Resources

Use curated/verified resources such as MDN, freeCodeCamp, Microsoft Learn, AWS Skill Builder, roadmap.sh and YouTube. Do not let the LLM invent URLs from memory; resources should come from seeded data or a real search mechanism.

17. Practice

Node.js example: Create Express server → Create CRUD API → Connect PostgreSQL → Add authentication → Handle API errors.

18. Project Recommendation

Example: Secure Task Management API → Node.js, Express, REST APIs, Authentication, PostgreSQL, Testing.

The project should create concrete evidence for the targeted skill gap.

19. Project Verification

Student submits a GitHub repository URL and identifies the skill the project is intended to demonstrate. SkillTwin analyzes the repository and returns evidence-backed verification.

README ✓ | Required concept ✓ | Authentication ✓ | Testing ✗ | Deployment ✓ → Verified / Needs Improvement + reasoning.

20. SkillTwin Update / Loop Closure

Before: Node.js – Beginner – 61% confidence Learn → Practice → Build → Verify After: Node.js – Intermediate – 87% confidence Career readiness recalculated.

The user should see that the SkillTwin evolved because of newly verified evidence.

21. Seven Core MVP Screens

- 1. Onboarding — profile, education, target role, study time, learning preferences.
- 2. Evidence Upload — Resume upload + GitHub connection.
- 3. SkillTwin — skills, proficiency, confidence, evidence and reasoning.
- 4. Target Role — select the target career.
- 5. Skill Gap — current vs required, severity, reasoning and priority.
- 6. Roadmap — Learn → Practice → Build with resources.
- 7. Project Verification — GitHub project → verify → update SkillTwin.

22. Technology Stack

Layer	Technology
Frontend	React + TypeScript
Backend	FastAPI + Python
Database	PostgreSQL

Intelligence	LLM-based structured intelligence
Contracts	Shared TypeScript types + validated backend contracts

Follow the latest PRD architecture rather than mixing unnecessary backend stacks.

23. What the LLM Does

- Resume parsing
- GitHub / repository analysis
- Skill extraction
- Skill normalization
- Gap analysis
- Priority ranking
- Roadmap generation
- Project verification

LLM outputs should be structured JSON and validated by the backend. Malformed outputs should be handled safely and retried rather than blindly trusted.

24. Database Concepts

- Users
- Evidence Sources
- Skills
- Skill Twin
- Skill Evidence Mapping
- Industry Roles
- Role Requirements
- Gap Reports
- Learning Resources
- Roadmaps
- Projects
- Assessments

25. Intended Repository Structure

```
skilltwin/ ■■■ backend/ ■■■ main.py ■■■ database.py ■■■ schema.sql ■■■ shared/ ■■■
llm_client.py ■■■ models.py ■■■ routers/ ■■■ evidence.py ■■■ roadmap.py ■■■
requirements.txt ■■■ frontend/ ■■■ src/ ■■■ App.tsx ■■■ modules/ ■■■ evidence/ ■■■
■■■ roadmap/ ■■■ shared/ ■■■ types.ts ■■■ apiClient.ts ■■■ docker-compose.yml ■■■
README.md
```

26. MVP — Must Build

- Student onboarding
- Resume upload
- Resume skill extraction

- GitHub analysis
- Skill normalization
- Living SkillTwin
- Target role selection
- Industry skill requirements
- Skill gap analysis
- Explainable gap reasoning
- Priority ranking
- Personalized roadmap
- Learning resources
- Practice exercises
- Project recommendation
- GitHub project verification
- SkillTwin update
- Career readiness / industry alignment

27. Strong Features — After Core MVP

- Skill adjacency / skill graph
- Job Description Analyzer
- What-If Career Simulator
- AI Skill Assessment
- Verified Skill Passport

28. Phase 2 / Future

- Adaptive DSA Coach with code execution
- What-If Career Simulator
- Deep Code Verification / deeper architecture-level inspection

29. What We Will NOT Do for the MVP

- Train a custom ML model
- Build a custom NLP model
- Scrape the entire internet for jobs
- Pretend curated requirements represent every company
- Generate fake course URLs
- Build every feature before the core works
- Make an AI chatbot the main innovation
- Create dozens of disconnected dashboards

30. Master Implementation Phases

PHASE 0 — FOUNDATION

Repository structure; frontend; backend; PostgreSQL; environment variables; LLM client; schema; seed data; shared types; API contract.

PHASE 1 — ONBOARDING

Student profile; education; target role; study time; learning preferences.

PHASE 2 — EVIDENCE

Resume upload/extraction/analysis + GitHub connection/fetch/analysis/evidence extraction.

PHASE 3 — SKILLTWIN

Normalization; proficiency; confidence; evidence mapping; reasoning; dashboard.

PHASE 4 — INDUSTRY MAPPING

Target roles; role requirements; matching; industry alignment.

PHASE 5 — GAP ENGINE

Gap calculation; severity; explainability; priority ranking.

PHASE 6 — ROADMAP

Resources; practice; projects; personalized ordering.

PHASE 7 — VERIFICATION

GitHub project submission; repository analysis; concept verification; evidence extraction.

PHASE 8 — LOOP CLOSURE

Update SkillTwin; recalculate proficiency, confidence and alignment.

PHASE 9 — POLISH

UI; loading states; error handling; security; testing; README; deployment; demo.

PHASE 10 — OPTIONAL POWER FEATURES

JDA; skill graph; adjacency; What-If; assessments; passport; etc.

31. Development Rules

- Do not build random disconnected features.
- Do not optimize only for UI; every visible feature must have real behavior behind it.
- Before each phase, understand the requirements, data flow and backend behavior.
- Implement one phase and test it end-to-end before moving to the next.
- Use the PRD and submitted concept as the source of truth.
- Inspect existing code before deciding whether to keep, modify, rebuild or remove it.
- Do not patch random UI problems without understanding the underlying feature flow.
- Every AI result that affects a user's profile should be evidence-backed and explainable.
- Prefer real data and real integrations over fake demo outputs when the MVP requires them.
- Keep optional features out of the critical path until the end-to-end MVP works.

32. Final Demo Flow

Create profile → Target Role: Full-Stack Developer → Study time → Upload Resume → Connect GitHub → Build SkillTwin → View Industry Alignment → Inspect a skill → See required/current level, confidence, evidence and reasoning → Identify a critical gap → Open roadmap → Learn → Practice → Build project → Submit GitHub repository → Verify concepts → Update SkillTwin → Recalculate readiness → Show that the SkillTwin evolved.

Demo goal: Show a complete evidence → gap → roadmap → build → verification → updated SkillTwin cycle, not merely attractive dashboard screens.

33. Master North Star

SkillTwin takes evidence of what a student has actually demonstrated, builds a living SkillTwin, compares it against the requirements of a target role, identifies and prioritizes skill gaps, provides a personalized path to close them, verifies newly acquired skills through projects, and updates the SkillTwin continuously.

Everything we build must serve this sentence.

34. What We Do Next

- Inspect the current repository and existing code.
- Compare the current implementation against this master plan.
- Classify existing work as KEEP / MODIFY / REBUILD / REMOVE.
- Start PHASE 0 only after understanding the current state.
- Complete each phase before moving forward.
- At every stage, test the real data flow—not just the appearance of the UI.

This PDF is the master working reference. Individual implementation details should remain consistent with the latest SkillTwin PRD and the submitted concept.